

Academy & Coaching Guide — Vermont Health Platform (HTR)

Audience: Faculty, content creators, product managers. **Version:** 4.2.0 **Route:** /academy

Table of Contents

- 1. Overview
- 2. Content Structure
- 3. Courses
- 4. Modules
- 5. Tracks
- 6. Webinars
- 7. Case Studies
- 8. Faculty
- 9. Glossary
- 10. Personalized Learning
- 11. Certificates
- 12. Academy Content in Sanity
- 13. Access & Gating

1. Overview

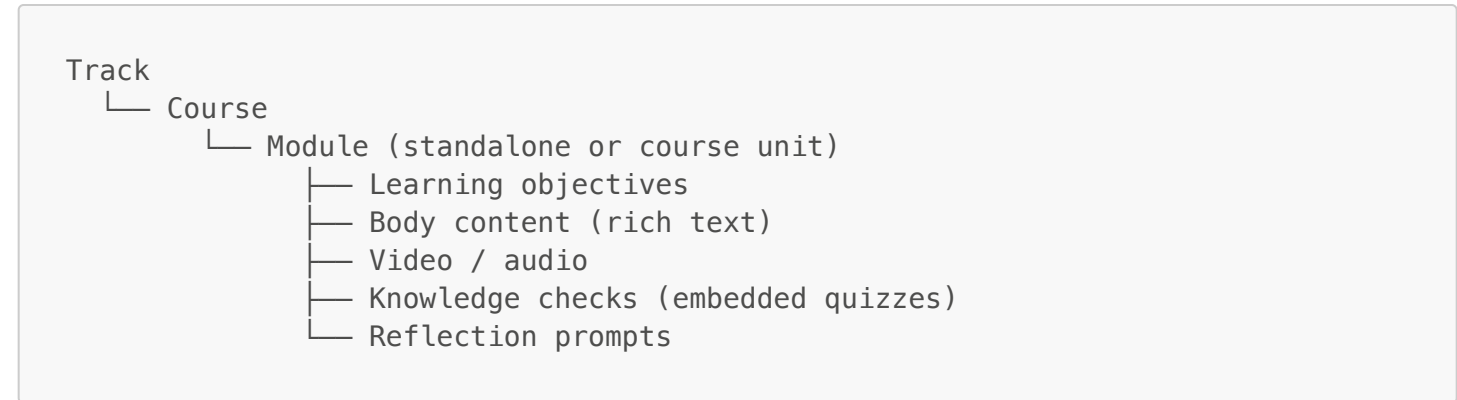
The HTR Academy is a structured learning management system for healthcare professionals navigating U.S. healthcare transformation. It provides:

- **Courses** — multi-module, graded, certifiable programs (typically 4–8 hours)
- **Modules** — standalone learning units (30–90 minutes each)
- **Tracks** — curated learning paths combining modules and courses around a theme
- **Webinars** — live and on-demand expert sessions
- **Case Studies** — real-world implementation analyses from health systems, payers, and states
- **Personalized Learning** — AI-generated individual curricula based on role and goals
- **Glossary** — 200+ healthcare transformation terminology definitions

Academy content is authored in Sanity CMS and rendered by the Next.js frontend. The AI Analyst knowledge base is indexed from all academy content, making modules and case studies queryable via chat.

2. Content Structure

Content Hierarchy



Modules can exist standalone (accessible individually) or as chapters within a Course. A Track groups Courses and standalone Modules into a coherent learning path.

Sanity Content Types

Type	Sanity Schema Name	Description
Module	academyModule	Standalone or course-chapter learning unit
Course	academyCourse	Multi-module program with enrollment
Track	academyTrack	Curated path of modules and courses
Webinar	webinar	Live or on-demand session
Case Study	caseStudy	Implementation analysis
Faculty	faculty	Instructor/expert profile
Glossary Term	definition	Term + definition + associated pillars

3. Courses

What Is a Course?

A Course is a structured multi-module program with:

- Defined learning objectives
- Sequential module chapters
- Graded knowledge assessments
- A certificate of completion upon passing

Courses are gated behind course enrollment, which requires a subscription tier that includes Academy access (Subscriber, Student, Professional, or Advisory).

Course Structure in Sanity

```
// academyCourse schema (key fields)
{
  title: string,
  slug: { current: string },
  pillar: "Policy" | "Economics" | "Technology" | "Clinical" | "Equity",
  difficulty: "Foundational" | "Intermediate" | "Advanced",
  estimatedHours: number,
  summary: string,
  learningObjectives: string[], // 3-5 objectives
  modules: Reference[], // ordered references to academyModule
  passingScore: number, // percentage (e.g. 80)
  certificateTemplate: Reference, // optional certificate design
  instructors: Reference[], // references to faculty
  prerequisites: Reference[], // other courses required first
}
```

Creating a Course

1. In Sanity Studio → Academy → Courses → Add
2. Complete all required fields: title, slug, pillar, difficulty, summary, learning objectives
3. Add modules in order (modules must be created first)
4. Set passing score (default: 80%)
5. Assign instructors from the Faculty list
6. Publish when ready

Enrollment Flow

When a user clicks "Enroll Now" on a course page, they are redirected to `/pricing?from=/academy/courses/[slug]`. After subscribing, they return to the course page and can begin.

Enrollment records are tracked in the `academy_enrollments` table (migration `003_academy.sql`).

4. Modules

What Is a Module?

A Module is the atomic unit of Academy learning. It can be:

- **Standalone** — directly accessible at `/academy/modules/[slug]`
- **Chapter** — part of a Course sequence, accessed through the course

Modules include rich text content, optional video/audio embeds, embedded knowledge checks, and reflection prompts.

Module Structure in Sanity

```
// academyModule schema (key fields)
{
  title: string,
  slug: { current: string },
  pillar: string,
  difficulty: "Foundational" | "Intermediate" | "Advanced",
  estimatedMinutes: number, // 30-90 typical
  summary: string,
  learningObjectives: string[],
  body: PortableText, // rich text with embeds
  videoUrl: string, // optional YouTube/Vimeo embed
  audioUrl: string, // optional MP3 URL
  knowledgeChecks: {
    question: string,
    options: string[],
    correct: number, // index of correct option
    explanation: string,
  }[],
  reflectionPrompts: string[],
  relatedModules: Reference[],
}
```

Writing a Module

Before writing:

- Identify the specific learning objective the module addresses
- Confirm the module fits within a pillar and difficulty level
- Check the glossary — link terms that have definitions

Body content guidelines:

- Open with a brief framing paragraph (why this matters)
- Use H2 headings for major sections, H3 for subsections
- Bullet points for lists of 3+ items
- Use the `callout` block type for key takeaways or warnings
- Embed video at the top if a lecture video exists
- Close with a 2-3 sentence summary of key insights

Knowledge checks:

- 3-5 questions per module
- One correct answer per question
- Explanation field required (shown after answer)
- Test comprehension, not memorization

5. Tracks

What Is a Track?

A Track is a curated learning path that sequences modules and courses to build mastery in a specific domain.
Example tracks:

- "VBC Fundamentals" — 5 modules, 1 course, ~8 hours total
- "AHEAD Model Deep Dive" — 3 modules focusing on Vermont's global budget model
- "Health Equity Analyst" — 4 modules + Health Equity Studio tool guidance

Track Structure in Sanity

```
// academyTrack schema (key fields)
{
  title: string,
  slug: { current: string },
  pillar: string,
  summary: string,
  targetAudience: string,
  estimatedHours: number,
  items: Array<{
    type: "module" | "course",
    reference: Reference,
    isOptional: boolean,
  }>,
  badges: string[], // competency badges earned on completion
}
```

Track Design Principles

- A track should have a clear audience (e.g., "Hospital CFOs new to value-based contracting")
- Order items progressively: foundational → intermediate → applied
- Mix content types: modules for concept, case studies for application
- Total time should be 4–12 hours for a meaningful track
- Mark 1–2 items as optional for learners who want to go deeper

6. Webinars

Webinars are live or on-demand expert sessions. They are managed in Sanity and rendered at </academy/webinars>.

Webinar Schema

```
// webinar schema (key fields)
{
  title: string,
  slug: { current: string },
  pillar: string,
  status: "upcoming" | "live" | "on-demand",
  scheduledAt: datetime, // for upcoming/live sessions
  durationMinutes: number,
  speakers: Reference[], // references to faculty
  summary: string,
  registrationUrl: string, // external link for upcoming sessions
  recordingUrl: string, // YouTube/Vimeo for on-demand
  slides: file, // optional PDF slides
  relatedContent: Reference[], // related modules, articles
}
```

Webinar Workflow

1. Create webinar in Sanity with `status: "upcoming"` and `scheduledAt` date
2. Add to the Webinars page — it will appear in the upcoming section
3. After the session, upload the recording URL and change `status` to `"on-demand"`
4. Update the webinar record with any slide uploads
5. Link related modules and articles so attendees can continue learning

7. Case Studies

Case studies analyze real-world healthcare transformation implementations. They are accessible at `/academy/case-studies` and are indexed by the AI Analyst.

Case Study Schema

```
// caseStudy schema (key fields)
{
  title: string,
  slug: { current: string },
  pillar: string,
  organization: string,           // health system, payer, state, etc.
  programType: string,          // "ACO", "Global Budget", "VBC Contract",
  etc.
  summary: string,
  body: PortableText,
  keyOutcomes: {
    metric: string,
    baseline: string,
    result: string,
    timeframe: string,
  }[],
  lessonsLearned: string[],
  relatedModules: Reference[],
}
```

Writing a Case Study

Structure:

1. **Context** — Organization background, market, why they undertook this initiative
2. **Approach** — What they did, key decisions, partners, timeline
3. **Results** — Quantified outcomes with baseline and timeframe
4. **Lessons Learned** — What worked, what didn't, what they'd do differently
5. **Applicability** — Who else can use this approach and under what conditions

Use the `keyOutcomes` structured field for quantified results — these render as a metrics card in the UI.

8. Faculty

Faculty profiles appear on course pages, webinar pages, and the `/academy/faculty` directory.

Faculty Schema

```
// faculty schema (key fields)
{
  name: string,
  slug: { current: string },
  title: string,                 // professional title
  organization: string,
```

```
bio: PortableText,
photo: image,
expertise: string[],           // pillar areas
credentials: string[],        // MD, JD, PhD, MHA, etc.
linkedIn: url,
}
```

Adding Faculty

1. Sanity Studio → Academy → Faculty → Add
2. Upload a professional headshot (minimum 400×400px, square crop preferred)
3. Write a 2–3 paragraph bio covering background, expertise, and HTR connection
4. Add expertise tags matching the five pillars
5. Publish and assign to relevant courses and webinars

9. Glossary

The Glossary at </academy/glossary> provides definitions for 200+ healthcare transformation terms. All glossary terms are indexed by the AI Analyst.

Glossary Schema

```
// definition schema (key fields)
{
  term: string,
  slug: { current: string },
  description: string,           // plain text definition (200–400 chars)
  pillars: string[],            // associated pillars for filtering
  relatedTerms: Reference[],    // other definition references
  relatedContent: Reference[],  // modules, articles that use this term
}
```

Adding Terms

- Write definitions for a non-expert audience first — use plain language
- Avoid jargon in the definition of jargon
- The **description** field is plain text (not rich text) — keep it to 200–400 characters
- Link 2–3 related terms to build concept connections
- Tag with the relevant pillar(s) so the glossary filter works

10. Personalized Learning

Overview

</academy/personalized-learning> generates a custom multi-week learning curriculum using the AI backend. The curriculum is structured as a week-by-week plan with specific learning items (readings, case studies, knowledge checks, reflections) drawn from HTR's content library.

User Flow

1. User fills out the onboarding form:
 - Role (Hospital Administrator, Policy Analyst, Clinician, etc.)
 - Topics of interest (multi-select from pillar areas)
 - Difficulty level (Foundational / Intermediate / Advanced)
 - Available hours per week
 - Learning goals (free text)
 - Format preferences (readings, case studies, knowledge checks)

2. Frontend sends the form data to `POST /api/personalized-learning`
3. Backend calls the LLM with a structured prompt to generate a JSON curriculum
4. The curriculum is returned and stored in `user_learning_paths`
5. User sees their week-by-week plan with progress tracking

Curriculum Structure

```
interface Curriculum {
  weeks: Week[];
}

interface Week {
  week: number;
  theme: string;
  focus_areas: string[];
  items: LearningItem[];
}

interface LearningItem {
  id: string;
  type: "reading" | "case_study" | "reflection" | "knowledge_check";
  title: string;
  description: string;
  content: string;
  platform_link: string;           // URL to relevant HTR page
  relevance_bridge?: string;       // why this is relevant to user's role
  estimated_minutes: number;
  key_concepts: string[];
  reflection_question?: string;
  questions?: KnowledgeCheckQuestion[];
}
```

Re-generating a Plan

Users can re-generate their plan at any time. The new plan overwrites the previous one in `user_learning_paths`.

11. Certificates

Certificate Issuance

A certificate is issued when a user completes a course with a passing score (default 80%). The certificate record is stored in the `academy_certificates` table (migration `003_academy.sql`) with:

- User ID
- Course ID
- Issue date
- Expiration date (if applicable — most certificates do not expire)
- A unique verification hash

Verification

Certificates are publicly verifiable at `/verify/[hash]`. The page displays:

- Holder's name
- Course name and description
- Issue date
- Expiration (if applicable)
- HTR seal and signature

The `/verify/` route is rate-limited to 20 requests/IP/60s (middleware-level) to prevent scraping.

Certificate Design

Certificates use an HTML-to-PDF template rendered server-side. The template is configured in `frontend/app/verify/[hash]/page.tsx` and uses the holder's name, course title, and issue date.

12. Academy Content in Sanity

Content Sync with AI

All published academy content (`academyModule`, `academyCourse`, `caseStudy`, `definition`) is indexed by the AI Analyst's RAG pipeline. When you publish or update any academy content, a Sanity webhook triggers a re-ingest. The AI Analyst will reflect the new content within ~15 minutes.

GROQ Queries Used for Indexing

The backend fetches academy content using these queries (from `backend/services/indexing.py`):

```
academyModule: title, pillar, summary, learningObjectives, bodyText (pt::text)
caseStudy: title, pillar, summary, bodyText (pt::text)
definition: term, description, pillars
```

Course-level content is not directly indexed (modules are). Ensure all key content is in the module body, not only in the course-level summary.

Image Guidelines

- Module hero images: 1200×630px minimum, 16:9 aspect ratio
- Faculty photos: 400×400px minimum, square crop
- Case study org logos: SVG preferred, 200px wide minimum
- All images are served via Sanity's CDN with automatic WebP conversion

13. Access & Gating

Content Access by Tier

Content Type	Free	Subscriber	Student	Professional	Advisory
Glossary	Full	Full	Full	Full	Full
Module previews	First 3 paragraphs	Full	Full	Full	Full
Module full content	—	Yes	Yes	Yes	Yes
Course enrollment	—	Yes	Yes	Yes	Yes
Personalized Learning	—	Yes	Yes	Yes	Yes
Webinar registration	—	Yes	Yes	Yes	Yes
Case studies	Preview	Full	Full	Full	Full
Certificates	—	Yes	Yes	Yes	Yes

Route Protection

The `/academy` route itself is publicly accessible (course listings, faculty, glossary index). Individual module and course content pages check the user's role server-side and either render full content or show an `UpgradePrompt` component after the 3-paragraph preview.

The personalized learning route (`/academy/personalized-learning`) requires Subscriber or above — enforced in `middleware.ts`.